

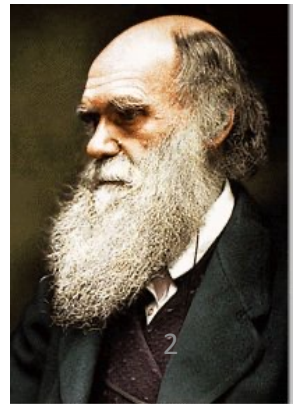
Computación Evolutiva

Clase 2

Instructor: Dr. Efrén Mezura Montes

Introducción

- La idea de simular la evolución dentro de una computadora no es nueva
- De hecho, derivadas de la misma idea, se han generado diversas áreas dentro de la computación
- Los Algoritmos Evolutivos (EAs por sus siglas en inglés) agrupan a aquellos basados en fenómenos relacionados con la evolución de las especies y la supervivencia del más apto
- A continuación se presenta una perspectiva histórica, escrita por De Jong (2006)



Los primeros años

- En 1930, Wright sugirió la utilidad de visualizar un sistema evolutivo que explora los picos de funciones multimodales mediante clusters alrededor de ellos (**visión hacia la optimización**)
- Friedman (1956) evolucionó circuitos de control para robots (**visión hacia el mecanismo de control de retroalimentación**)
- Friedberg (1959) propuso una “máquina que aprende” y que evoluciona conjuntos de instrucciones de lenguaje de máquina (**visión hacia la programación automática**)

Los catalíticos 1960's

- Tres grupos definieron la forma de este campo emergente:
- En la U. T. de Berlín, Rechenberg y Schwefel formularon ideas sobre cómo un proceso evolutivo puede usarse para resolver problemas numéricos de optimización complejos. De estas ideas surge un paradigma conocido como “**Estrategias Evolutivas**”



Los catalíticos 1960's

- En UCLA Fogel visualizó a la evolución como un medio para alcanzar las metas de la IA, por lo que evolucionó agentes inteligentes representados como máquinas de estado finito. De ello se derivó el paradigma conocido como **“Programación Evolutiva”**



Los catalíticos 1960's

- En la U. de Michigan, Holland vió a un proceso evolutivo como la clave en el diseño e implementación de sistemas adaptativos robustos, capaces de lidiar con un ambiente cambiante e incierto. Remarcó la necesidad de los sistemas de auto-adaptarse al interaccionar con el ambiente. Sus ideas originaron los “Planes reproductivos”, mejor conocidos como el paradigma de los **“Algoritmos Genéticos”**



Los catalíticos 1960's

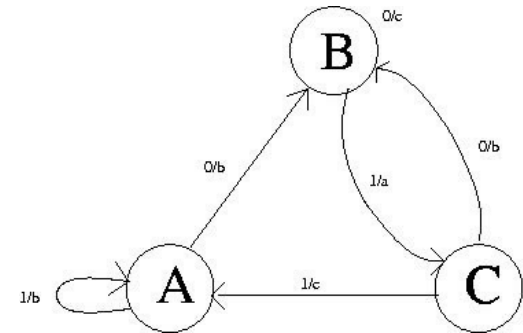
- Estos paradigmas, más que reflejar fielmente los procesos evolutivos, son modelos “ideales” de evolución orientados a la resolución de problemas
- Sin embargo, un análisis formal de su funcionamiento es altamente complejo (se requiere de simplificar el modelo)

Los explorativos 1970's

- Derivado de las propuestas de la década previa, dos aspectos quedaron sin resolver:
 - Caracterizar la implementación de los paradigmas
 - Entender su aplicabilidad
- En esta década se avanzó en estos dos temas vía:
 - Estudios empíricos
 - Teoría

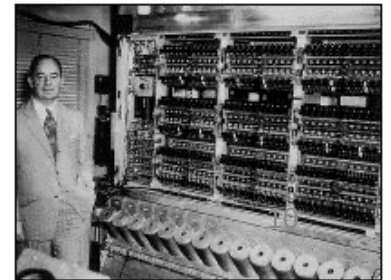
Los explorativos 1970's

- Se establecen con ello, tres EAs:
 - Programación evolutiva
 - Estrategias evolutivas
 - Algoritmos genéticos
- **Evolutionary programming (EP)** se concentró en modelos con reproducción principalmente asexual y reemplazo probabilístico, siendo parámetros críticos el tamaño de población y el porcentaje de reproducción asexual



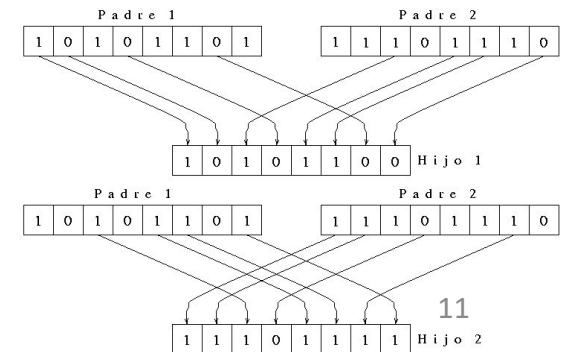
Los explorativos 1970's

- **Evolution strategies (ES)** se centraron en la optimización numérica (números reales), los modelos iniciales $(1+1)$ -ES y $(1+\lambda)$ -ES usaron reproducción asexual usando una distribución Gaussiana y reemplazo determinístico. Sin embargo, lo que distingue a las ES es el proceso de adaptación o auto-adaptación de sus parámetros relativos a la mutación.



Los explorativos 1970's

- **Genetic Algorithms (GAs)** promovieron algoritmos “independientes de la aplicación”, mediante una representación de cadenas de bits, uso intensivo de reproducción sexual, reemplazo generacional y selección basada en aptitud. Los “teoremas de los esquemas” conforman contribuciones a nivel teórico del desempeño de los GAs.



Los explotativos 1980's

- De los 1970's emergió una colección de EAs canónicos con amplia aplicabilidad y desempeño altamente competitivo
- En los 1980's se amplió la diversidad de aplicaciones y se generaron variantes de EAs específicas para otros dominios

Los explotativos 1980's

- $(\mu+\lambda)$ -ES, (μ,λ) -ES
- Selección proporcional, representaciones alternativas y decepción en GAs
- Conocimiento “a-priori” del problema (P.Ej. Operadores especiales)
- Además de la optimización, surgen dominios de aplicación como los sistemas clasificadores, redes neuronales, redes de tareas, código en Lisp
- EAs paralelos
- Surgen los primeros congresos: ICGA'85, 87 y 89, PPSN'90, FOGA'90

Los unificadores 1990's

- Hacia finales de los 1980's, el trabajo de cada EA se hacía por separado
- El surgimiento de los primeros congresos permitió el trabajo de colaboración y diferenciación, además del acuñamiento del término “Cómputo Evolutivo, EC” y del inicio de trabajos del primer journal “Evolutionary Computation” del MIT press
- Cruzamiento de mecanismos entre EAs

El siglo XXI, expansión madura

- Se tienen 3 journals de prestigio:
 - Evolutionary Computation
 - IEEE Transactions on Evolutionary Computation
 - Genetic Programming and Evolvable Machines
- Conferencias de alto nivel
 - CEC
 - GECCO
 - PPSN
 - FOGA

El siglo XXI, expansión madura

- Mucho trabajo por hacer:
 - Aplicaciones de alto impacto
 - Estudios teóricos
 - Ajuste de parámetros
 - Co-evolución
 - Sinergias con Machine Learning y Agent Systems
 - Consolidación del área

¿Por qué EC?

- La búsqueda de solucionadores de problemas es tarea de la ciencia de la computación
- Los solucionadores “naturales” por excelencia son:
 - El cerebro humano (quien creó la rueda, ciudades, la guerra, etc.)
 - El proceso evolutivo (quien creó el cerebro humano)

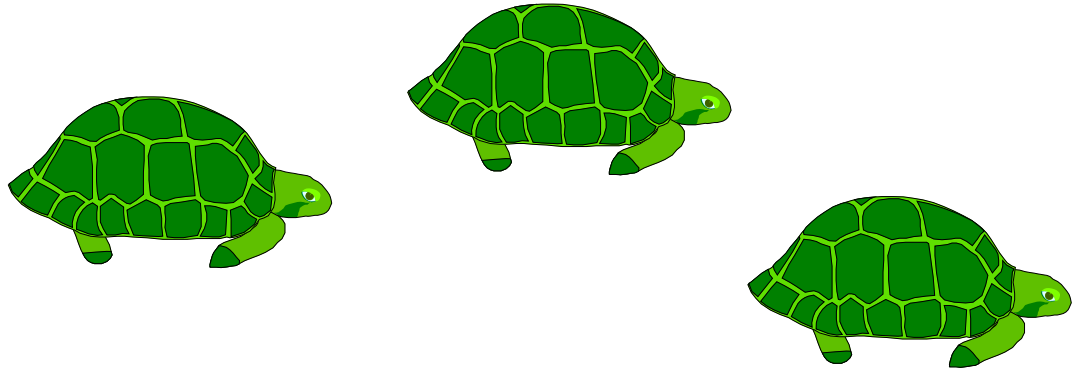
¿Por qué EC?

- Por otro lado, se requiere de solucionadores aplicables a una amplia gama de problemas, sin requerir grandes cambios para problemas específicos, que devuelvan buenas soluciones en un tiempo razonable



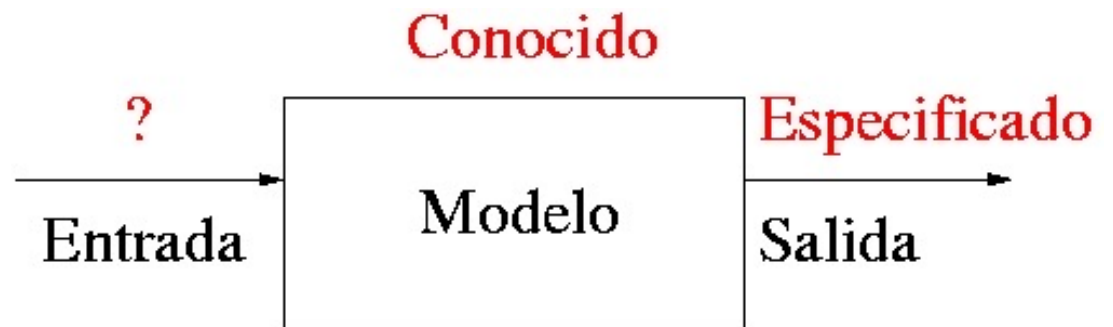
¿Por qué EC?

- Finalmente, se puede utilizar la simulación de la evolución para generar procesos difíciles de realizar físicamente, además de que este tipo de simulaciones pueden ayudar a entender fenómenos naturales



Tipos de problemas

- Optimización
 - Consiste en encontrar la entrada adecuada para la salida que se especifique
- Ejemplo: Diseño óptimo de pieza mecánica



Un ejemplo

- Diseño de una pieza tubular

- Variables

- Largo (l)
- Diámetro (d)
- Grosor (g)

- Condiciones

$$100\text{cm} \leq l \leq 1000\text{cm}$$

$$10\text{cm} \leq d \leq 50\text{cm}$$

$$1\text{cm} \leq g \leq 5\text{cm}$$

- Objetivo

- Minimizar el peso $f(l, d, g) = \pi \left(\frac{d}{2} \right)^2 l + 0.35g^3$



¿Cómo solucionarlo?

- Probar combinaciones

- $l=100, d=10, g=1$

- $l=100, d=10, g=2$

- $l=100, d=10, g=3$

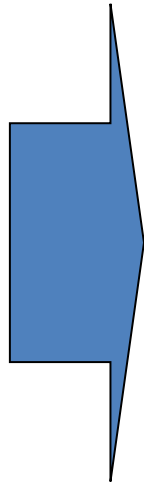
- $l=100, d=10, g=4$

- $l=100, d=10, g=5$

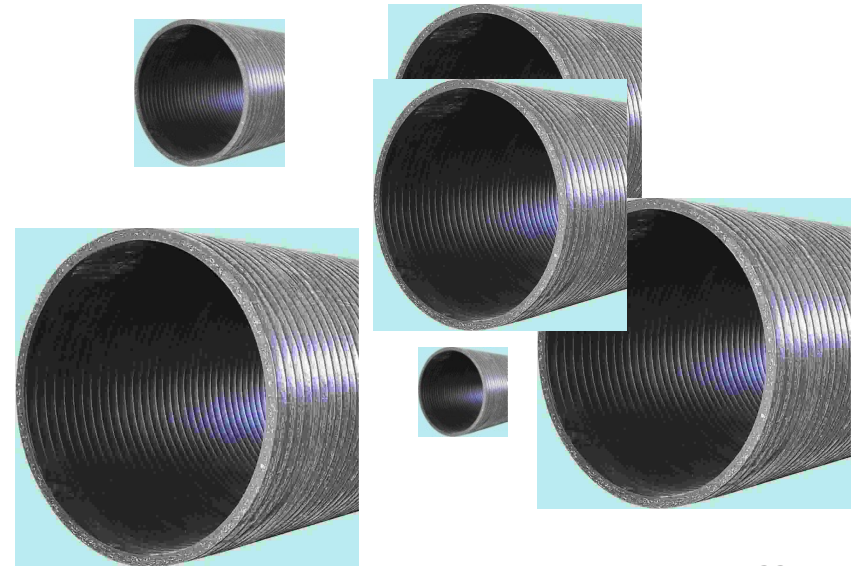
- .

- .

- .



$$f(l, d, g) = \pi \left(\frac{d}{2} \right)^2 l + 0.35 g^3$$

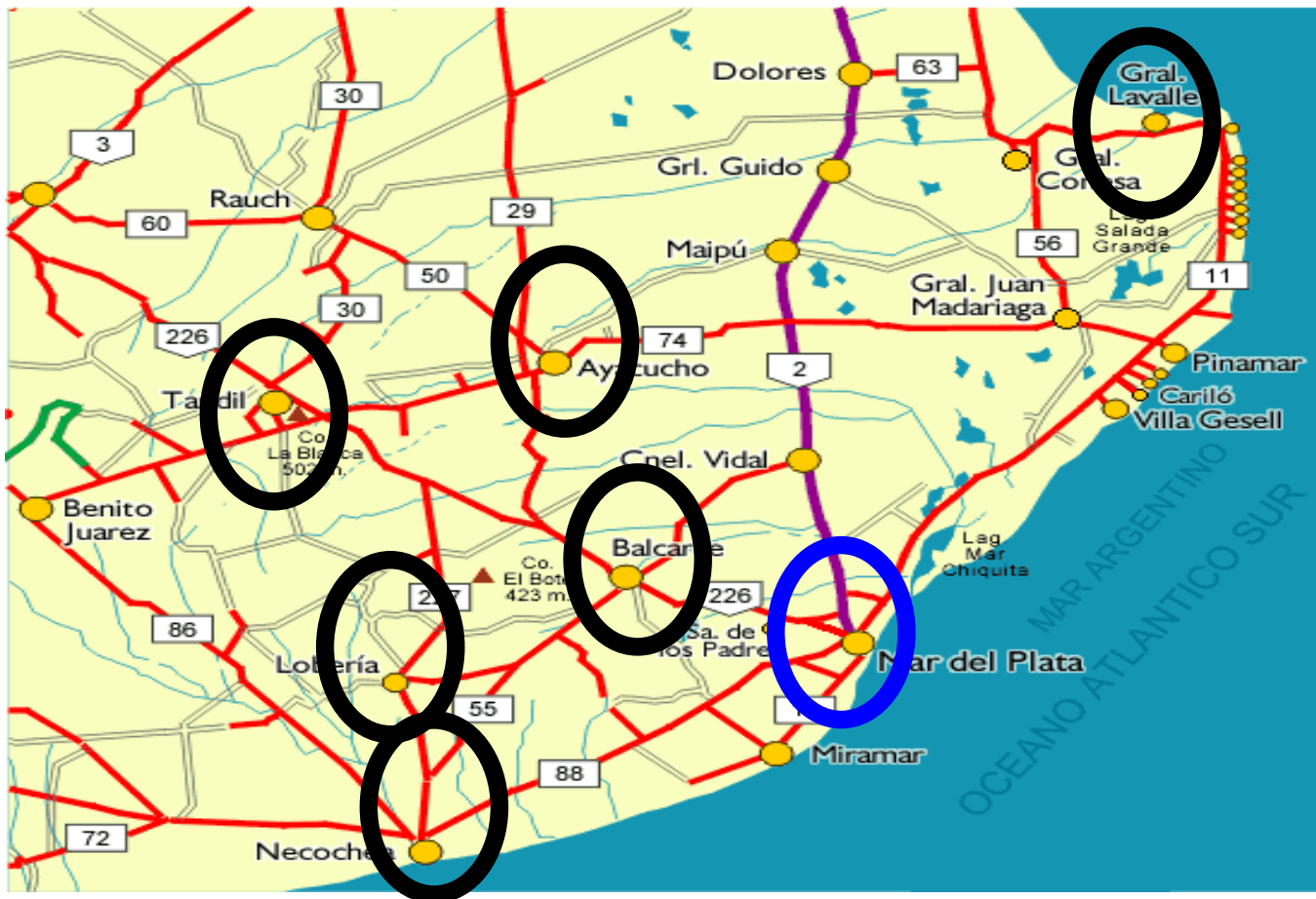


¿Cuántas soluciones hay?

- Sol. de “l” X Sol. de “d” X Sol. de “g”
 $100\text{cm} \leq l \leq 1000\text{cm}$
 $10\text{cm} \leq d \leq 50\text{cm}$
- $901 \times 41 \times 4 = 147\,764$ soluciones
 $1\text{cm} \leq g \leq 5\text{cm}$
- Si agregamos 2 decimales de exactitud a cada solución ...
 - $(901 \times 10^2) \times (41 \times 10^2) \times (4 \times 10^2) = 1.44 \times 10^{11}$
 - $\approx 147,764,000,000$ soluciones

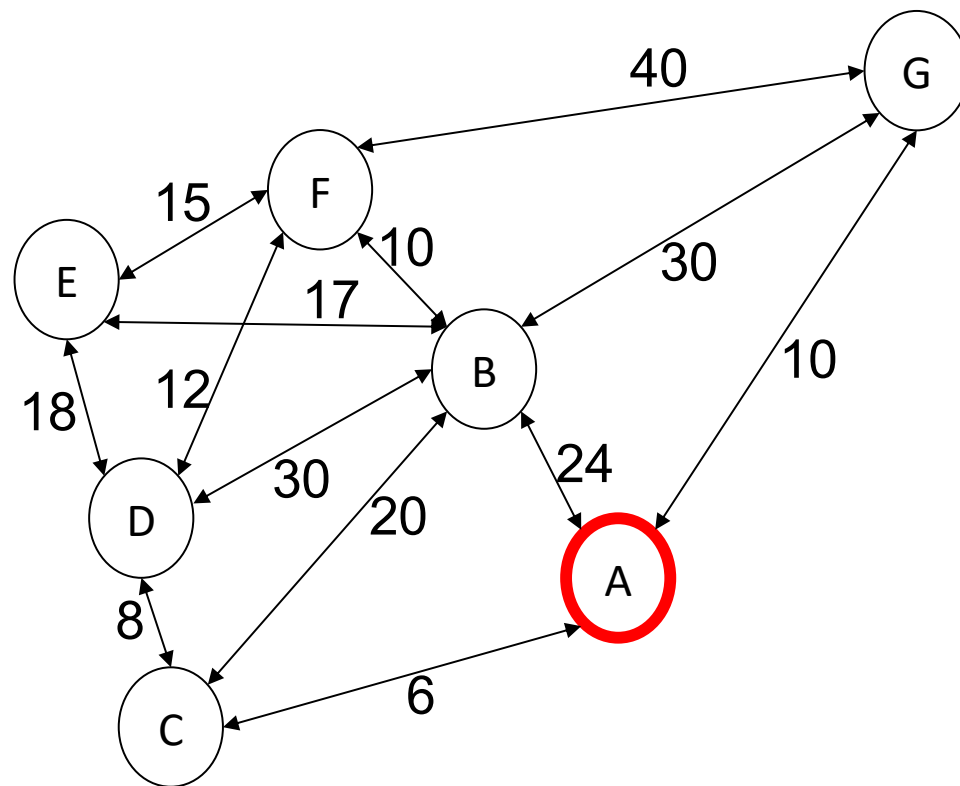
Otro ejemplo

- Diseño de rutas para empresas de mensajería



¿Cómo lo resolvemos?

- Grafo dirigido



¿Cuántas soluciones hay?

- Encontrar una permutación que represente el recorrido de un conjunto de ciudades de tal forma que todas sean visitadas una sola vez, minimizando la distancia recorrida .
- El espacio de búsqueda crece de acuerdo al número de ciudades:

$$(n-1)!/2$$

¿Cuántas soluciones hay?

- Para $n=10$, hay 181,000 soluciones posibles.
- Para $n=20$, hay 10,000,000,000,000,000 soluciones posibles.
- Para $n=50$ hay
100,000,000,000,000,000,000,000,000,000,000,
0,000,000,000,000,000,000,000,000,000,000,000
soluciones posibles.
- Hay 1,000,000,000,000,000,000,000 litros de agua en el planeta (aproximadamente).

Los problemas con los problemas del mundo real

[Michalewicz. 2004]

- El número de soluciones posibles puede llegar a ser prohibitivo para una búsqueda exhaustiva
- El problema es muy complicado y sólo se pueden utilizar modelos simplificados del mismo
- La función objetivo puede variar con respecto al tiempo
- Las soluciones están altamente restringidas

Clasificación de problemas

- Optimización
 - Optimización paramétrica
 - Optimización con restricciones
 - Optimización de estructuras de datos
- Satisfacción de restricciones
- Aprendizaje de Máquina
- Programación automática
- Optimización dinámica

Técnicas clásicas

- La Investigación de operaciones es la parte de las matemáticas que ofrece y aplica técnicas para resolver problemas de este tipo
 - Programación lineal
 - Programación entera
 - Programación no lineal
 - Programación dinámica
 - Toma de decisiones multicriterio

Ventajas

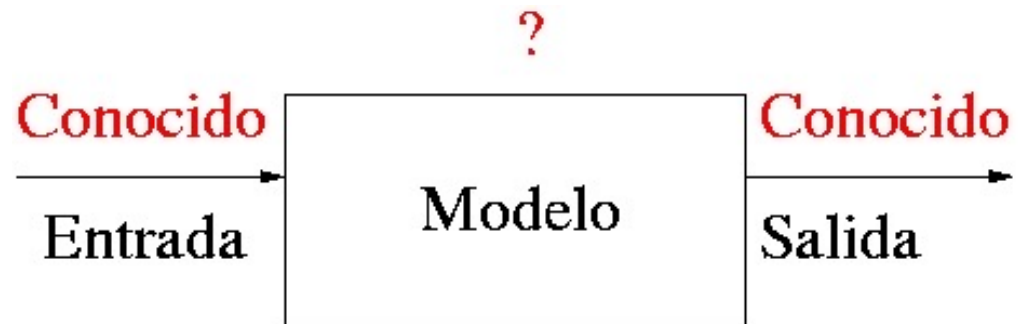
- Son la primera opción para resolver un problema de búsqueda/optimización
- Si las características del problema se ajustan a las condiciones requeridas por el método, pueden garantizar encontrar la mejor solución
- El costo computacional puede ser bajo en condiciones adecuadas

Desventajas

- En algunos problemas no pueden ser aplicadas o pueden tardar mucho tiempo en devolver una solución aceptable
- Pueden estancarse en soluciones óptimas locales
- La aplicación puede requerir la transformación del problema original
- Algunos métodos distan de ser sencillos de entender y/o aplicar

Tipos de problemas

- Modelado
 - Dadas las entradas y las salidas, se pretende encontrar el modelo que proporcione el comportamiento indicado
- Ejemplo: Reconocimiento de caracteres



Tipos de problemas

- Simulación
 - La tarea consiste en procesar las salidas correspondientes a partir de un conjunto de entradas conocidas y un modelo establecido

- Ejemplo: Circuito electrónico para filtrado

